

8. Effacer sans compter les lettres

Le compilateur connaît la longueur du texte. Autant qu'il la compte.

1. Ce qu'il faut comprendre

Écrire des espaces par-dessus **est** l'effacement — l'espace est la tuile 0. Mais il en faut **autant que de lettres** : un de moins laisse la dernière orpheline, un de plus mange la case d'à côté. Ni l'un ni l'autre n'est signalé, puisque écrire des espaces est parfaitement légal.

Or le compilateur **connaît** cette longueur : il la lit dans les guillemets, ou dans la table nommée. La compter soi-même, c'était se donner une occasion de se tromper que rien n'obligeait à prendre.

`effacer(colonne, ligne, quoi)` accepte les **trois** façons de dire combien : le texte lui-même, le nom d'un `const char`, ou un nombre — calculé s'il le faut. Et `effacer(..., 0)` n'efface **aucune** case, ce qui n'allait pas de soi : sur un octet, décompter à partir de zéro donne 255.

`effacerPanneau()` fait de même sur le panneau — sans argument il le vide entièrement, à trois arguments il n'ôte qu'un mot.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **594 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         101 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases, EcrireTexte - attend de pouvoir écrire une case
EffacerCases   12 o ← effacer() - efface des cases
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
CopierTuiles   58 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         256 o ← CopierTuiles - 16 lettres sur 43
"GAME OVER"    9 o ← texte() - un texte gravé
"BONJOUR"      7 o ← texte() - un texte gravé
"ABCDEF"       6 o ← texte() - un texte gravé
"JE RESTE"     8 o ← texte() - un texte gravé
```

Lettres gravées : ABCDEFGJMNORSTUV

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte()   × 4    44 o sur place, dans main() + EcrireTexte (10 o)
effacer() × 3    25 o sur place, dans main() + EffacerCases (12 o)
image()   × 2     6 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : écrit quatre textes, attend un moment, puis en efface trois
// avec effacer() – sans avoir à compter les lettres soi-même.
//
// effacer(colonne, ligne, QUOI) écrit des espaces à la place d'un texte :
//   avec un texte entre guillemets : autant d'espaces que de lettres ;
//   avec un nombre : ce nombre d'espaces.

#include <texte>      // écrit un texte à l'écran
#include <effacer>    // efface des cases, ou tout le fond
```

2. UN TEXTE NOMMÉ — FIN

```
const char FIN[] = "GAME OVER"; // Un texte qui porte un nom : FIN.
                                // const char ...[] = un texte gravé qui ne change pas.
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    texte(2, 4, "BONJOUR");                  // Quatre textes, sur les lignes 4, 6, 8 et 10.
    texte(2, 6, FIN);                        // On peut écrire un texte par son nom.
    texte(2, 8, "ABCDEF");
    texte(2, 10, "JE RESTE");

    for (uint8_t i = 0; i < 90; i++) image(); // Attend 90 images = une seconde et demie.

    effacer(2, 4, "BONJOUR"); // sept cases, comptées pour nous
    effacer(2, 6, FIN);       // neuf, lues dans la table nommée
    uint8_t combien = 3;      // Un nombre choisi pendant le jeu...
    effacer(2, 8, combien);    // trois, décidées à l'exécution : ABCDEF devient DEF
                                // JE RESTE, lui, n'est pas effacé.

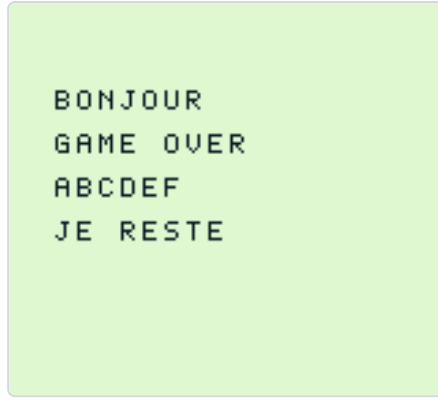
    while (true) {                // Répète pour toujours :
        image();                  // attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

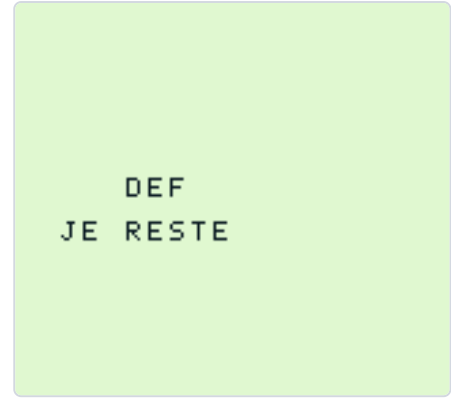
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Trois lignes s'effacent au bout d'une seconde et demie ; « JE RESTE » ne bouge pas, et « DEF » survit.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le texte est là au départ
- ✓ `effacer("...")` ôte exactement sa longueur
- ✓ un texte nommé aussi
- ✓ un nombre calculé n'ôte que ce nombre
- ✓ et rien d'autre n'est touché

Ensuite : **8.1. Chapitre 2 — Les variables : la mémoire de la console : ce qui vient** — chapitre 2 : Les variables : la mémoire de la console. · `gameboy3` — écrire une cartouche Game Boy en C++